

图上的结构算法

不要背七段代码：追踪候选域、提交时刻与仍然成立的事实

408 · 数据结构 Topic DS-08

母问题：图上的“下一步”凭什么不会破坏全局目标？

所有算法先压缩为同一条生成链：

Goal → Candidate Domain → Local Action → Commit Point → Invariant → Global Result

最容易混淆的不是循环语法，而是“候选对象是谁、何时可以永久确定”。Prim 与 Kruskal 都求最小生成树，却分别在单棵树边界与全图森林之间选择；Dijkstra、Bellman–Ford 与 Floyd 都做松弛，却分别固定顶点、路径边数上界与允许经过的中间点集合。

本册定位与 Owner 边界

- **Owns**：Prim/Kruskal、Dijkstra/Bellman–Ford/Floyd、拓扑排序与 AOE 关键路径的目标、不变量、提交条件和失败证据。
- **Uses**：DS07 提供图表示与遍历状态；Kruskal 直接复用 DS06 Union-Find；Prim/Dijkstra 复用 DS05 的“按优先级取候选”抽象契约。
- **接口事实**：当前 Prim 用数组扫描，Dijkstra 用标准库最小优先队列，并未直接复用 DS05 的整数最大堆类；不能把概念依赖写成代码依赖。
- **Stops**：堆、并查集和图存储内部机制不在本册重讲；工程路由协议只可调用最短路模型。

目录

1	先认候选域，再认算法名	3
2	最小生成树：同一目标，两种生长几何	3
2.1	Prim：一棵树向外吞并	3
2.2	Kruskal：多棵树逐步合并	4
3	最短路：同一个松弛式，三种“已经解决”	5
3.1	Dijkstra：固定当前最近的顶点	5
3.2	Bellman–Ford：固定路径允许使用的边数	6
3.3	Floyd：固定允许经过的中间点集合	7
4	DAG：从释放约束到识别零时差活动	9
4.1	拓扑排序：删除的不是点，而是前置约束	9
4.2	关键路径：正向求最早，反向求最迟	9
5	测试不是报喜，而是攻击不变量	11
6	复杂度与适用边界	13

7 考场与代码复原协议

13

1 先认候选域，再认算法名

算法	目标对象	候选域	提交时刻	失败证据
Prim	一棵 MST	当前树到树外的割边	树外顶点以最小跨边权加入	仍有顶点却无有限割边
Kruskal	一棵 MST	全图尚未处理的边	最小边连接两个不同分量	处理完仍不足 $V - 1$ 条边
Dijkstra	单源最短路	未过期的暂定距离	最小暂定距离出队	出现负权边
Bellman-Ford	单源最短路	每一轮的全部有向边	第 i 轮保证至多 i 条边的最短路	$V - 1$ 轮后仍可松弛
Floyd	全源最短路	所有点对 (i, j)	中间点集合由前 $k - 1$ 个扩为前 k 个	最终存在 $d[i][i] < 0$
拓扑排序	约束合法序列	当前入度为零的顶点	顶点出队并释放后继约束	输出顶点数小于 V
关键路径	AOE 最长工期与零时差活动	拓扑序上的事件和活动	正向、反向两遍完成后	图有环或工期为负判时差

统一记忆锚点

不要记“某算法总是取最小”。先补完整句子：从哪个候选域，按什么量取谁；取出后是暂时用于传播，还是从此永久确定？只要候选域或提交时刻答错，代码即使长得像模板，证明也已经失效。

2 最小生成树：同一目标，两种生长几何

2.1 Prim：一棵树向外吞并

设 S 为已经进入树的顶点集合， $\text{best}[v]$ 表示从 S 跨到顶点 v 的最轻边权。每轮选择树外 best 最小的顶点加入 S ，再用它的邻边缩小其他顶点的 best 。

循环不变量

已提交顶点始终连成一棵树；每个未提交顶点的 best 是当前割 $(S, V - S)$ 上到该点的最轻已知边。割性质保证本轮最轻跨边是安全边。若树外仍有顶点而所有 best 都是无穷大，证据不是“程序没找到”，而是图不连通。

Listing 1: Prim：候选域是当前树的边界

```

32 inline Distance prim_weight(std::size_t vertex_count, const std::vector<
    Edge>& edges) {
33     if (vertex_count == 0) {
34         return 0;
35     }
36     std::vector<std::vector<std::pair<std::size_t, int>>> graph(
        vertex_count);
37     for (const Edge& edge : edges) {
38         check_edge(vertex_count, edge);
39         graph[edge.from].push_back({edge.to, edge.weight});
    
```

```

40     graph[edge.to].push_back({edge.from, edge.weight});
41 }
42
43 std::vector<Distance> best_crossing_edge(vertex_count, kInfinity);
44 std::vector<bool> fixed(vertex_count, false);
45 best_crossing_edge[0] = 0;
46 Distance total = 0;
47 for (std::size_t step = 0; step < vertex_count; ++step) {
48     std::size_t chosen = vertex_count;
49     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
50         if (!fixed[vertex] &&
51             (chosen == vertex_count || best_crossing_edge[vertex] <
52              best_crossing_edge[chosen])) {
53             chosen = vertex;
54         }
55     }
56     if (chosen == vertex_count || best_crossing_edge[chosen] == kInfinity)
57     {
58         throw std::invalid_argument("graph is disconnected");
59     }
60     fixed[chosen] = true;
61     total += best_crossing_edge[chosen];
62     for (const auto [next, weight] : graph[chosen]) {
63         if (!fixed[next] && weight < best_crossing_edge[next]) {
64             best_crossing_edge[next] = weight;
65         }
66     }
67 }
68 return total;
69 }

```

2.2 Kruskal: 多棵树逐步合并

Kruskal 不维护“当前树”，而维护无环森林。边按权从小到大扫描；只有两端属于不同连通分量时，才把边提交并合并分量。

同一张图为什么会产生两种轨迹

边为 01(1), 12(2), 23(1), 03(8)。从顶点 0 出发的 Prim 轨迹是

$$\{0\} \rightarrow \{0, 1\} \rightarrow \{0, 1, 2\} \rightarrow \{0, 1, 2, 3\};$$

Kruskal 则可先同时形成分量 $\{0, 1\}$ 与 $\{2, 3\}$ ，再用 12(2) 合并。两者总权都为 4，但状态几何完全不同：Prim 是单树扩张，Kruskal 是森林合并。

Listing 2: Kruskal: Union-Find 维护森林分量

```

69 inline Distance kruskal_weight(std::size_t vertex_count, std::vector<Edge
    > edges) {
70     for (const Edge& edge : edges) {

```

```

71     check_edge(vertex_count, edge);
72 }
73 std::sort(edges.begin(), edges.end(),
74           [](const Edge& left, const Edge& right) { return left.weight <
75               right.weight; });
76 ds06::UnionFind components(vertex_count);
77 Distance total = 0;
78 std::size_t chosen_edges = 0;
79 for (const Edge& edge : edges) {
80     if (components.unite(edge.from, edge.to)) {
81         total += edge.weight;
82         ++chosen_edges;
83         if (chosen_edges + 1 == vertex_count) {
84             break;
85         }
86     }
87 }
88 if (vertex_count > 0 && chosen_edges + 1 != vertex_count) {
89     throw std::invalid_argument("graph is disconnected");
90 }
91 return total;
92 }

```

3 最短路：同一个松弛式，三种“已经解决”

对边 (u, v, w) ，三种算法共享局部动作

$$d[v] \leftarrow \min\{d[v], d[u] + w\},$$

但局部动作相同不代表外层循环可以互换。外层循环决定每一步之后究竟固定了什么。

3.1 Dijkstra：固定当前最近的顶点

非负权保证未来绕路只会继续增加长度。因此，当未过期的最小暂定距离 (d, u) 出队时，不存在一条经未确定顶点绕回来使 $d[u]$ 更小的路径。

读代码时只盯三个状态

1. `distance[v]`：目前最好上界，不等于所有时刻都已最终确定；
2. `pending`：可能含同一顶点的旧候选；
3. `current_distance != distance[current]`：丢弃过期候选，只有未过期最小项才承担提交语义。

负权边会破坏“取出即确定”的证明，所以实现必须在运行前拒绝它，而不是算完再碰碰运气。

Listing 3: Dijkstra：非负权、最小候选与过期项过滤

```

93 inline std::vector<Distance> dijkstra(std::size_t vertex_count,
94                                       const std::vector<Edge>& edges,

```

```

95         std::size_t source) {
96     if (source >= vertex_count) {
97         throw std::out_of_range("source vertex out of range");
98     }
99     std::vector<std::vector<std::pair<std::size_t, int>>> graph(
100         vertex_count);
101     for (const Edge& edge : edges) {
102         check_edge(vertex_count, edge);
103         if (edge.weight < 0) {
104             throw std::invalid_argument("dijkstra requires nonnegative edges");
105         }
106         graph[edge.from].push_back({edge.to, edge.weight});
107     }
108     using Candidate = std::pair<Distance, std::size_t>;
109     std::priority_queue<Candidate, std::vector<Candidate>, std::greater<
110         Candidate>> pending;
111     std::vector<Distance> distance(vertex_count, kInfinity);
112     distance[source] = 0;
113     pending.push({0, source});
114     while (!pending.empty()) {
115         const auto [current_distance, current] = pending.top();
116         pending.pop();
117         if (current_distance != distance[current]) {
118             continue;
119         }
120         for (const auto [next, weight] : graph[current]) {
121             const Distance candidate = current_distance + static_cast<Distance>(
122                 weight);
123             if (candidate < distance[next]) {
124                 distance[next] = candidate;
125                 pending.push({candidate, next});
126             }
127         }
128     }
129     return distance;
130 }

```

3.2 Bellman–Ford: 固定路径允许使用的边数

第 i 轮扫描全部边后，可以保证所有“至多含 i 条边”的最短路已被传播。没有可达负环时，简单最短路至多含 $V - 1$ 条边；因此 $V - 1$ 轮后若仍能松弛，就得到一个可达负环证据。

最常漏掉的保护条件

若 $d[u] = \infty$ ，边 $u \rightarrow v$ 不可参与加法，更不能据此更新 v 。“无穷大不参与算术”同时是语义规则和防溢出规则。

Listing 4: Bellman–Ford: 轮数就是路径边数上界

```

130 inline std::vector<Distance> bellman_ford(std::size_t vertex_count,
131                                           const std::vector<Edge>& edges,
132                                           std::size_t source) {
133     if (source >= vertex_count) {
134         throw std::out_of_range("source vertex out of range");
135     }
136     for (const Edge& edge : edges) {
137         check_edge(vertex_count, edge);
138     }
139     std::vector<Distance> distance(vertex_count, kInfinity);
140     distance[source] = 0;
141     for (std::size_t round = 1; round < vertex_count; ++round) {
142         bool changed = false;
143         for (const Edge& edge : edges) {
144             if (distance[edge.from] == kInfinity) {
145                 continue;
146             }
147             const Distance candidate = distance[edge.from] + static_cast<
Distance>(edge.weight);
148             if (candidate < distance[edge.to]) {
149                 distance[edge.to] = candidate;
150                 changed = true;
151             }
152         }
153         if (!changed) {
154             break;
155         }
156     }
157     for (const Edge& edge : edges) {
158         if (distance[edge.from] != kInfinity &&
159             distance[edge.from] + static_cast<Distance>(edge.weight) <
distance[edge.to]) {
160             throw std::invalid_argument("reachable negative cycle");
161         }
162     }
163     return distance;
164 }

```

3.3 Floyd: 固定允许经过的中间点集合

令 $D^{(k)}[i][j]$ 表示只允许前 k 个顶点作为中间点时, i 到 j 的最短距离, 则

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]).$$

因此 middle 必须放在最外层: 完成第 k 层之后, 整张矩阵才具有统一的“允许中间点集合”语义。

Listing 5: Floyd: 最外层循环扩张允许中间点集合

```

166 inline std::vector<std::vector<Distance>> floyd_warshall(
167     std::size_t vertex_count, const std::vector<Edge>& edges) {
168     std::vector<std::vector<Distance>> distance(
169         vertex_count, std::vector<Distance>(vertex_count, kInfinity));
170     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
171         distance[vertex][vertex] = 0;
172     }
173     for (const Edge& edge : edges) {
174         check_edge(vertex_count, edge);
175         distance[edge.from][edge.to] =
176             std::min(distance[edge.from][edge.to], static_cast<Distance>(edge
177                 .weight));
178     }
179     for (std::size_t middle = 0; middle < vertex_count; ++middle) {
180         for (std::size_t from = 0; from < vertex_count; ++from) {
181             if (distance[from][middle] == kInfinity) {
182                 continue;
183             }
184             for (std::size_t to = 0; to < vertex_count; ++to) {
185                 if (distance[middle][to] == kInfinity) {
186                     continue;
187                 }
188                 distance[from][to] = std::min(
189                     distance[from][to], distance[from][middle] + distance[middle
190                         ][to]);
191             }
192         }
193     }
194     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
195         if (distance[vertex][vertex] < 0) {
196             throw std::invalid_argument("graph contains a negative cycle");
197         }
198     }
199     return distance;
200 }

```

三算法一眼分流

- **Dijkstra**: 固定顶点；需要非负权；适合单源。
- **Bellman-Ford**: 固定路径边数上界；允许负边；可检测源点可达负环。
- **Floyd**: 固定允许的中间点集合；处理所有点对；对角线负值暴露负环。

看到“松弛”还不能选算法，必须再问：题目要求固定哪一种状态？

4 DAG：从释放约束到识别零时差活动

4.1 拓扑排序：删除的不是点，而是前置约束

入度为零表示一个顶点的全部前置约束已经释放。顶点出队后，对每条出边将后继入度减一；只有减到零的后继才进入候选队列。若队列耗尽但仍有顶点未输出，这些顶点之间存在无法释放的循环依赖。

Listing 6: Kahn 拓扑排序：候选域是当前零入度点

```

200 inline std::vector<std::size_t> topological_order(
201     std::size_t vertex_count, const std::vector<Edge>& edges) {
202     std::vector<std::vector<std::size_t>> graph(vertex_count);
203     std::vector<std::size_t> indegree(vertex_count, 0);
204     for (const Edge& edge : edges) {
205         check_edge(vertex_count, edge);
206         graph[edge.from].push_back(edge.to);
207         ++indegree[edge.to];
208     }
209     std::queue<std::size_t> ready;
210     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
211         if (indegree[vertex] == 0) {
212             ready.push(vertex);
213         }
214     }
215     std::vector<std::size_t> order;
216     while (!ready.empty()) {
217         const std::size_t current = ready.front();
218         ready.pop();
219         order.push_back(current);
220         for (const std::size_t next : graph[current]) {
221             if (--indegree[next] == 0) {
222                 ready.push(next);
223             }
224         }
225     }
226     if (order.size() != vertex_count) {
227         throw std::invalid_argument("directed graph contains a cycle");
228     }
229     return order;
230 }

```

4.2 关键路径：正向求最早，反向求最迟

AOE 网中，顶点是事件，边是有持续时间的活动。沿拓扑序正向做最长路松弛：

$$ve[v] = \max_{u \rightarrow v} \{ve[u] + w(u, v)\}.$$

项目工期是所有事件最早发生时间的最大值。再以工期初始化 vl ，沿逆拓扑序反向收紧：

$$vl[u] = \min_{u \rightarrow v} \{vl[v] - w(u, v)\}.$$

对活动 $u \rightarrow v$ ，其最早开始与最迟开始分别为

$$e = ve[u], \quad l = vl[v] - w(u, v).$$

只有 $e = l$ ，即总时差为零，活动才是关键活动。

为什么只算正向最长路还不够

正向一遍只能回答“项目最早多久完成”；它不能判断某条活动能否延迟而不影响总工期。关键活动判断必须依赖反向的最迟时间。代码若只返回 `earliest`，就没有完成关键路径算法。

Listing 7: 关键路径：最早、最迟与零时差边

```

232 struct CriticalPathResult {
233     Distance duration = 0;
234     std::vector<Distance> earliest_event;
235     std::vector<Distance> latest_event;
236     std::vector<std::size_t> critical_edge_indices;
237 };
238
239 inline CriticalPathResult critical_path(std::size_t vertex_count,
240                                         const std::vector<Edge>& edges) {
241     for (const Edge& edge : edges) {
242         check_edge(vertex_count, edge);
243         if (edge.weight < 0) {
244             throw std::invalid_argument("activity duration must be nonnegative"
245                                         );
246         }
247     }
248     const std::vector<std::size_t> order = topological_order(vertex_count,
249                                                             edges);
250     CriticalPathResult result;
251     result.earliest_event.assign(vertex_count, 0);
252     for (const std::size_t current : order) {
253         for (const Edge& edge : edges) {
254             if (edge.from == current) {
255                 result.earliest_event[edge.to] =
256                     std::max(result.earliest_event[edge.to],
257                             result.earliest_event[current] + edge.weight);
258             }
259         }
260     }
261     for (const Distance time : result.earliest_event) {
262         result.duration = std::max(result.duration, time);
263     }
264     result.latest_event.assign(vertex_count, result.duration);
265     for (auto current = order.rbegin(); current != order.rend(); ++current)
266     {
267         for (const Edge& edge : edges) {
268             if (edge.from == *current) {

```

```

266         result.latest_event[edge.from] =
267             std::min(result.latest_event[edge.from],
268                     result.latest_event[edge.to] - edge.weight);
269     }
270 }
271 }
272 for (std::size_t index = 0; index < edges.size(); ++index) {
273     const Edge& edge = edges[index];
274     const Distance earliest_start = result.earliest_event[edge.from];
275     const Distance latest_start = result.latest_event[edge.to] - edge.
weight;
276     if (earliest_start == latest_start) {
277         result.critical_edge_indices.push_back(index);
278     }
279 }
280 return result;
281 }

```

5 测试不是报喜，而是攻击不变量

Listing 8: 同目标对照、失败边界与完整关键路径断言

```

29 void test_mst_algorithms_share_the_same_goal() {
30     const std::vector<Edge> edges{{0, 1, 1}, {1, 2, 2}, {2, 3, 1}, {0, 3,
      8}};
31     assert(prim_weight(4, edges) == 4);
32     assert(kruskal_weight(4, edges) == 4);
33     assert_invalid([&] { (void)prim_weight(5, edges); });
34     assert_invalid([&] { (void)kruskal_weight(5, edges); });
35 }
36
37 void test_shortest_path_algorithms_and_boundaries() {
38     const std::vector<Edge> nonnegative{
39         {0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 5}};
40     const std::vector<Distance> expected{0, 3, 1, 4, kInfinity};
41     assert(dijkstra(5, nonnegative, 0) == expected);
42     assert(bellman_ford(5, nonnegative, 0) == expected);
43
44     const std::vector<Edge> with_negative{{0, 1, 4}, {1, 2, -3}, {0, 2,
      5}};
45     assert((bellman_ford(3, with_negative, 0) == std::vector<Distance>{0,
      4, 1}));
46     assert_invalid([&] { (void)dijkstra(3, with_negative, 0); });
47     assert_invalid([&] { (void)bellman_ford(2, {{0, 1, -1}, {1, 0, -1}}, 0)
      ; });
48
49     const auto all_pairs = floyd_warshall(4, nonnegative);

```

```

50  assert(all_pairs[0][3] == 4);
51  assert(all_pairs[3][0] == kInfinity);
52  assert_invalid([&] { (void)floyd_warshall(2, {{0, 1, -1}, {1, 0, -1}});
    });
53
54  const std::vector<Edge> large_weights{{0, 1, 2'000'000'000}, {1, 2, 2'
    000'000'000}};
55  assert(dijkstra(3, large_weights, 0)[2] == 4'000'000'000LL);
56 }
57
58 void test_topology_and_critical_path() {
59     const std::vector<Edge> activities{
60         {0, 1, 3}, {0, 2, 2}, {1, 3, 4}, {2, 3, 3}, {2, 4, 2}, {3, 5, 2},
61         {4, 5, 3}};
62     const auto order = topological_order(6, activities);
63     assert(order.front() == 0 && order.back() == 5);
64
65     const auto result = critical_path(6, activities);
66     assert(result.duration == 9);
67     assert((result.earliest_event == std::vector<Distance>{0, 3, 2, 7, 4,
68         9}));
69     assert((result.latest_event == std::vector<Distance>{0, 3, 4, 7, 6, 9}
70         ));
71     assert((result.critical_edge_indices == std::vector<std::size_t>{0, 2,
72         5}));
73
74     assert_invalid([&] { (void)topological_order(2, {{0, 1, 1}, {1, 0, 1}})
75         ; });
76     assert_invalid([&] { (void)critical_path(2, {{0, 1, -1}}); });
77 }

```

测试按容易混淆的接口成对设计：Prim 与 Kruskal 在同图上应得到同一 MST 总权；Dijkstra 与 Bellman-Ford 在非负图上应得到相同单源距离；负边只拒绝 Dijkstra，负环才拒绝 Bellman-Ford；拓扑排序输出序列，关键路径还必须输出 ve 、 vl 与关键边。

路径代价使用 long long，并用两条 2×10^9 的边验证结果为 4×10^9 ，避免算法逻辑正确却被 32 位整数边界破坏。

6 复杂度与适用边界

算法	本实现复杂度	正确性前提	显式失败
Prim	$O(V^2 + E)$	无向连通带权图	非连通
Kruskal	$O(E \log E)$	无向连通带权图	非连通
Dijkstra	$O((V + E) \log V)$	有向图边权非负	负边
Bellman-Ford	$O(VE)$	允许负边	源可达负环
Floyd	$O(V^3)$	距离加法不溢出	任意负环
拓扑排序	$O(V + E)$	有向无环图	有环
关键路径	当前实现 $O(VE)$	AOE 网为 DAG, 工期非负	有环、负工期

7 考场与代码复原协议

遇到图算法题，按五步写在草稿上：

1. **Goal:** 要树、单源距离、全源距离、合法顺序，还是总工期与关键活动？
2. **Candidate:** 候选是跨割边、全局边、暂定距离、全部边、点对，还是零入度点？
3. **Commit:** 本轮结束后，哪个对象从“候选”变为“永久成立”？
4. **Update:** 更新距离、入度、分量、跨边权，还是最早/最迟时间？
5. **Failure:** 负边、负环、非连通、有环、不可达与溢出分别怎样显式出现？

压缩复原

只记五问：求什么？候选是谁？何时提交？提交后什么永真？什么证据让证明失效？当五问能在新图上独立回答，代码才真正被压缩成心智模型；只会背函数名和循环层数，还没有形成模型。

代码证据

完整实现位于 `code/ds08_graph_algorithms.hpp`，断言测试位于 `code/ds08_graph_algorithms_test.cpp`。正文中的行号引用与当前 C++17 实现同步。